ALFIDO TECH

Java Developer Internship

TASK 4 REPORT

Student Record File Manager

Submitted By: Alquama Arsalan Khan

Submission Date: 22 June 2026

ABSTRACT

This project presents the development of a Student Record File Manager using Java. The application is designed to demonstrate the implementation of file handling operations through a simple menu-driven interface. Users can store student records in a text file and retrieve them whenever required.

The project utilizes Java FileReader and FileWriter classes for reading and writing data, respectively. Additionally, it incorporates user input handling, exception management, and basic software development practices. The source code is maintained using GitHub, providing an introduction to version control and project documentation.

This project serves as a practical example of how file handling can be applied to manage data efficiently in Java applications.

OBJECTIVE

The primary objective of this project is to understand and implement file handling concepts in Java through the development of a console-based application. The project focuses on storing and retrieving student information using text files while ensuring efficient data management.

Furthermore, the project aims to strengthen programming fundamentals, improve problem-solving skills, and provide hands-on experience with software development tools such as Visual Studio Code and GitHub.

PROJECT OVERVIEW

The Student Record File Manager is a Java-based console application developed to manage student records through file operations. The application enables users to add student names and store them permanently in a text file. It also allows users to read and display previously saved records.

The system follows a menu-driven approach, making it simple and user-friendly. Through this project, essential Java programming concepts such as file handling, exception handling, loops, conditional statements, and user interaction have been implemented successfully.

SYSTEM FEATURES

Features Implemented

- Add Student Records
- Save Data Using FileWriter
- Read Data Using FileReader
- Interactive Menu-Driven Interface
- Exception Handling
- Data Persistence Through Files
- GitHub Repository Management

TECHNOLOGIES USED

Technology	Purpose
Java	Application Development
VS Code	Code Editor
Scanner Class	User Input
FileWriter	Writing Data to File

Technology	Purpose
FileReader	Reading Data from File
GitHub	Version Control & Hosting

WORKING OF THE PROJECT

Step 1

The application starts and displays a menu of available options.

Step 2

The user selects the desired operation.

Step 3

When the Add Student option is selected, the application accepts student information and stores it in a text file.

Step 4

When the Read Students option is selected, the application retrieves the stored records and displays them.

Step 5

The application continues running until the user chooses to exit.

CONCEPT USED

Scanner Class

The Scanner class is used to accept input from the user through the keyboard. It allows users to interact with the application by entering menu choices and student names.

Usage

- Taking menu choices.
- Accepting student names.

FileWriter

FileWriter is used to write data into a text file. It enables the application to store student records permanently.

Usage

- Saving student names into students.txt.
- Creating and updating file content.

Benefits

- Easy file writing.
- Permanent data storage.
- Efficient handling of text files.

FileReader

FileReader is used to read data from a text file. It retrieves stored student records and displays them to the user.

Usage

- Reading student records.
- Displaying file contents.

Benefits

- Simple file reading.

- Efficient retrieval of stored information.

Exception Handling

Exception handling is implemented using try-catch blocks to prevent program crashes and handle file-related errors effectively.

Usage

- Handling file reading errors.
- Handling file writing errors.

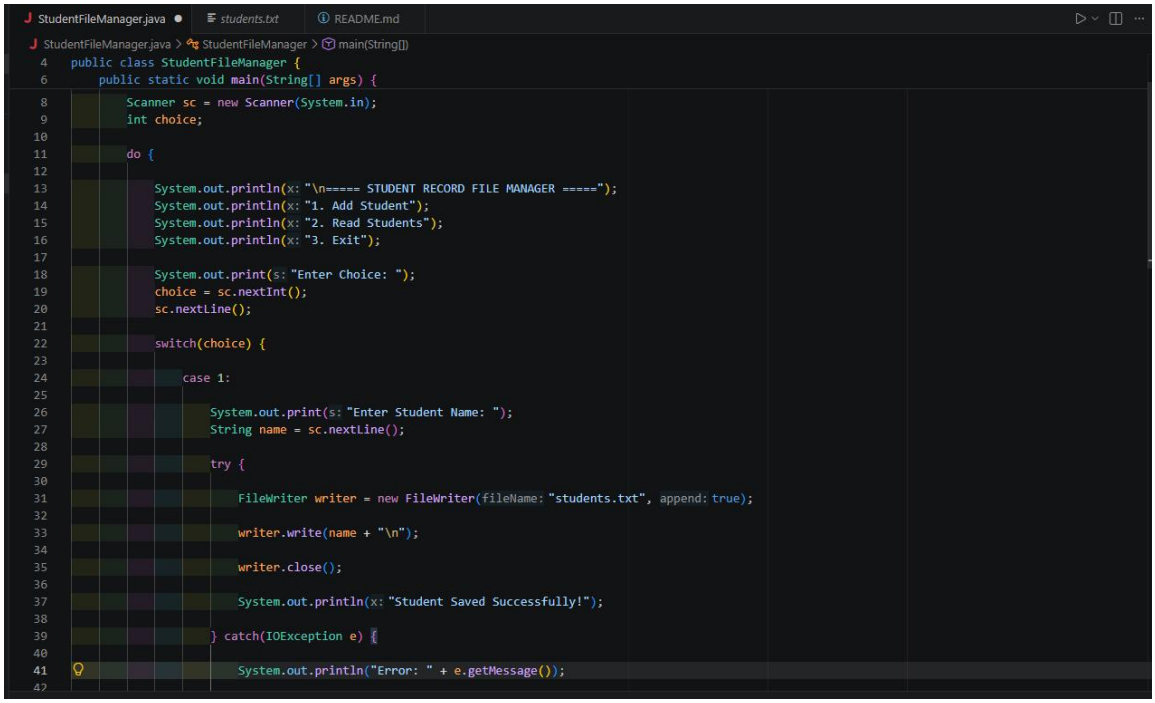
Menu-Driven Programming

The application follows a menu-driven approach that allows users to select operations from a list of options.

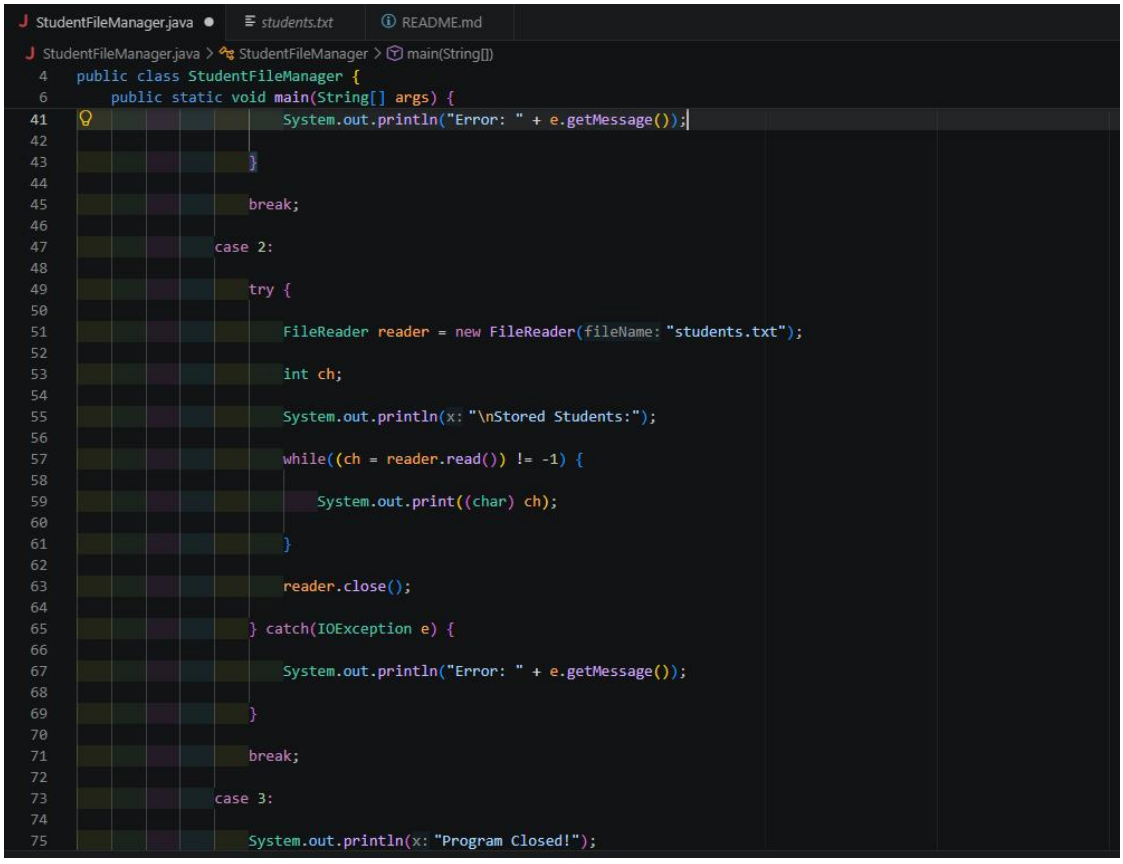
Benefits

- Easy navigation.
- User-friendly interaction.
- Better program organization.

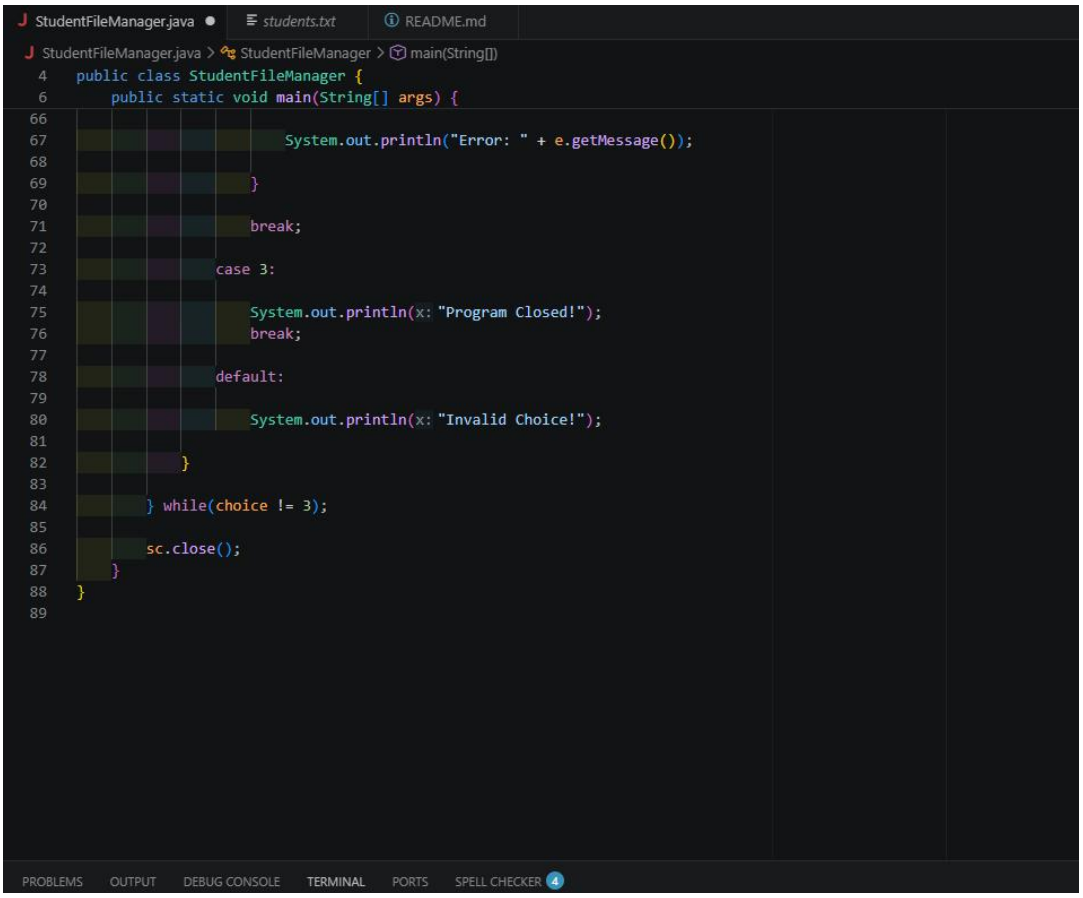
SOURCE CODE



```java
public class StudentFileManager {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {

            System.out.println("\n===== STUDENT RECORD FILE MANAGER =====");
            System.out.println("1. Add Student");
            System.out.println("2. Read Students");
            System.out.println("3. Exit");

            System.out.print("Enter Choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            switch(choice) {

                case 1:

                    System.out.print("Enter Student Name: ");
                    String name = sc.nextLine();

                    try {

                        FileWriter writer = new FileWriter("students.txt", true);

                        writer.write(name + "\n");

                        writer.close();

                        System.out.println("Student Saved Successfully!");

                    } catch(IOException e) {

                        System.out.println("Error: " + e.getMessage());
```



```java
                        System.out.println("Error: " + e.getMessage());
                    }

                    break;

                case 2:

                    try {

                        FileReader reader = new FileReader("students.txt");

                        int ch;

                        System.out.println("\nStored Students:");

                        while((ch = reader.read()) != -1) {

                            System.out.print((char) ch);

                        }

                        reader.close();

                    } catch(IOException e) {

                        System.out.println("Error: " + e.getMessage());

                    }

                    break;

                case 3:

                    System.out.println("Program Closed!");
```

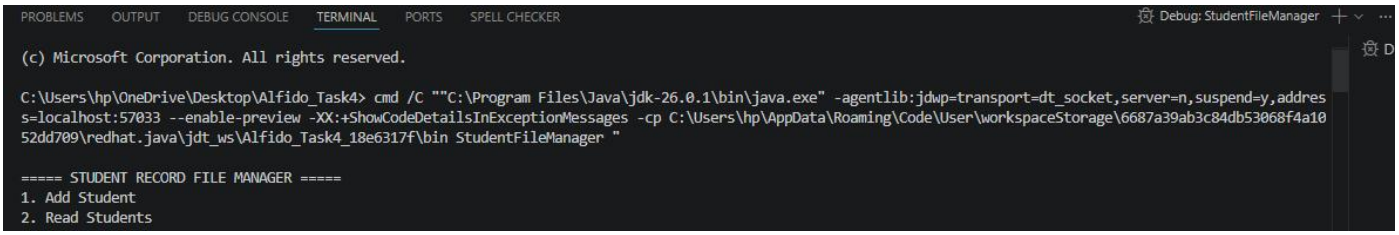

StudentFileManager.java
students.txt
README.md
StudentFileManager.java > StudentFileManager > main(String[])
4 public class StudentFileManager {
6 public static void main(String[] args) {
66
67 System.out.println("Error: " + e.getMessage());
68
69 }
70
71 break;
72
73 case 3:
74
75 System.out.println(x: "Program Closed!");
76 break;
77
78 default:
79
80 System.out.println(x: "Invalid Choice!");
81
82 }
83
84 } while(choice != 3);
85
86 sc.close();
87 }
88 }
89
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SPELL CHECKER 4

Sample Input/Output Screenshots

Screenshot 1: Main Menu

Description

The application starts by displaying a menu containing different options available to the user. The user can choose to add student records, read stored records, or exit the application.



Screenshot 2: Adding Student Records

Description

The user selects the Add Student option and enters the student name. The application stores the entered information in the students.txt file and displays a confirmation message indicating successful data storage.

Screenshot 3: Reading Student Records

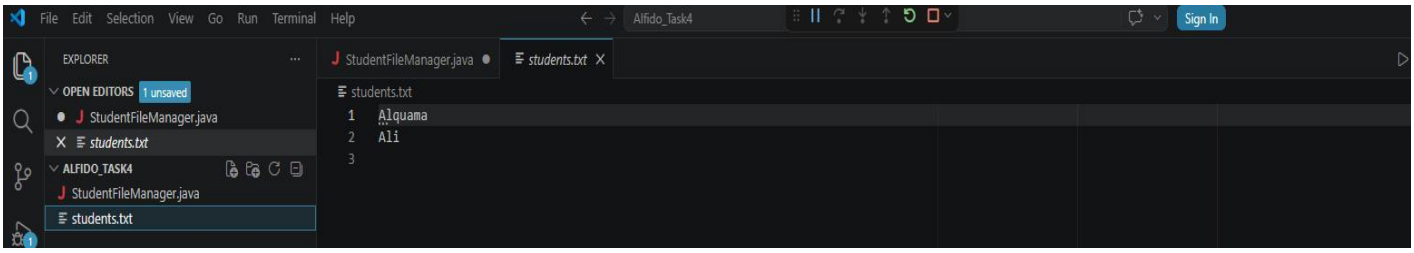
Description

The user selects the Read Students option. The application reads the stored student records from the file using FileReader and displays them on the console.

Screenshot 4: students.txt File

Description

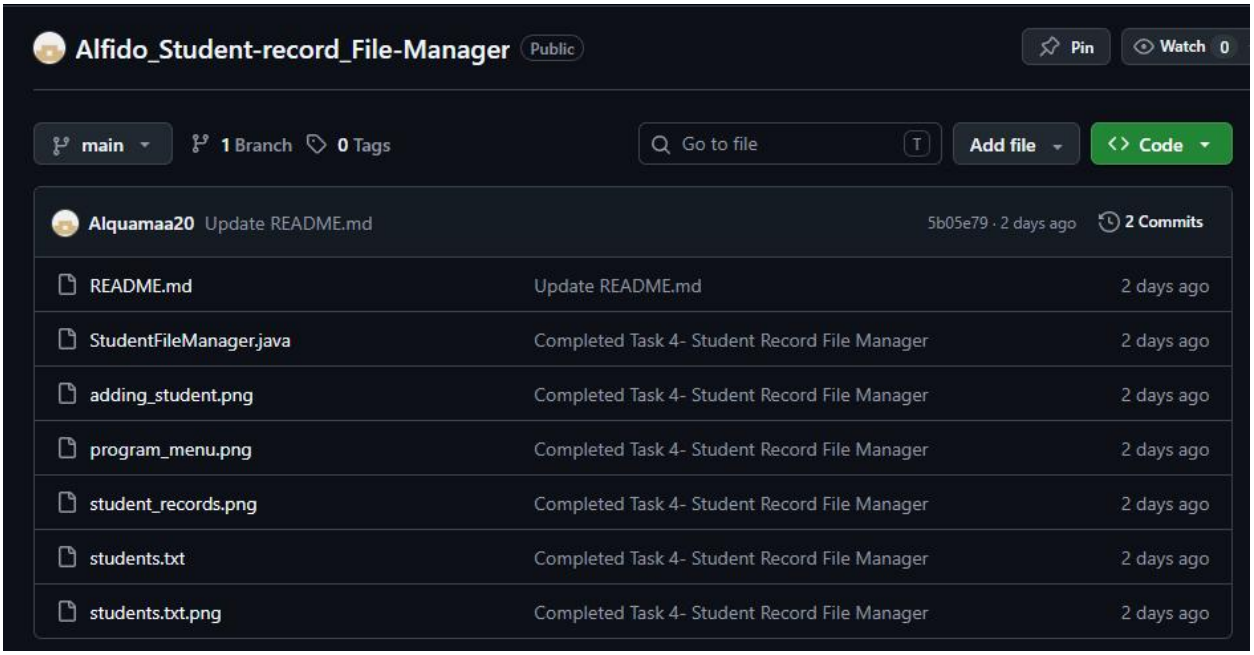
The students.txt file contains the student records that were saved by the application. This demonstrates successful file writing operations using FileWriter.



Screenshot 5: GitHub Repository

Description

The project source code and documentation were uploaded to GitHub for version control and project management. The repository contains the Java source code and README file.



LEARNING OUTCOMES

- Developed practical understanding of Java File Handling.
- Learned to use FileReader and FileWriter classes effectively.
- Gained experience in creating menu-driven applications.
- Improved debugging and problem-solving abilities.
- Learned the fundamentals of version control using GitHub.
- Enhanced software documentation skills.

CONCLUSION

The Student Record File Manager project successfully demonstrates the implementation of Java file handling operations. Through the use of FileReader and FileWriter, data can be stored and retrieved efficiently, providing a simple yet effective record management solution.

The project enhanced understanding of Java programming concepts, improved software development skills, and introduced version control practices through GitHub. Overall, the project achieved its objectives and provided valuable hands-on experience in application development.

GITHUB REPOSITORY

Repository Link:
https://github.com/Alquamaa20/Alfido_Student-record_File-Manager

REFERENCES

1. Oracle Java Documentation
2. Visual Studio Code Documentation
3. GitHub Documentation
4. Java Programming Tutorials
5. File Handling Concepts in Java